Informe de evidencias — Laboratorio 04

1. Identificación

Aprendiz: Karen Sirley Bonilla Ramirez
Ficha: 3388087
Laboratorio: Laboratorio 04 — Persistencia, archivos, excepciones y programación orientada a objetos
Repositorio: https://github.com/Bonillatalen/laboratorios

2. Descripción de las actividades

En este laboratorio continué el mismo proyecto de los laboratorios anteriores. Conservé las carpetas `laboratorio-01`, `laboratorio-02` y `laboratorio-03`, y agregué `laboratorio-04` sin reemplazar el trabajo anterior. También publiqué las prácticas del material en `practica-4-1`, `practica-4-2`, `practica-4-3` y `practica-4-4`, además del archivo `RESPUESTAS_COMPROBACION_Y_ANALISIS.md`.

La actividad principal fue convertir el gestor de inventario en una aplicación persistente. La solución quedó separada en varios módulos:

- `producto.py`: representa cada producto, encapsula sus datos y valida código, nombre, categoría, precio y cantidad.
- `inventario.py`: contiene las operaciones de negocio para registrar, consultar, buscar, actualizar, eliminar y resumir productos. También permite cargar registros de manera segura.
- `almacenamiento.py`: guarda y recupera el inventario en JSON, usa rutas relativas al módulo, valida el contenido y protege el archivo cuando el JSON está dañado.
- `reportes.py`: genera un informe CSV sin mezclar la exportación con la lógica del inventario.
- `main.py`: presenta el menú interactivo, maneja entradas y errores, y coordina guardado, recuperación y exportación.
- `test_persistencia.py`: comprueba las operaciones principales, la persistencia, el archivo dañado y el cálculo del reporte.

El programa permite registrar productos, listarlos, buscar por código, actualizar existencias, eliminarlos, consultar un resumen y exportar `inventario.csv`. El valor económico se calcula como `precio * cantidad`.

3. Estructuras, archivos y conceptos aplicados

Usé clases, propiedades, encapsulamiento, composición e instancias independientes. El inventario contiene productos, por lo que la relación correcta es de composición y no de herencia. También utilicé listas y diccionarios para organizar los registros, `Path` para construir rutas confiables, `try/except` para manejar errores de entrada y archivos, JSON para persistencia estructurada y CSV para reportes tabulares.

En las prácticas complementarias trabajé excepciones, archivos de texto, rutas, lectura y escritura, JSON, CSV, serialización, herencia, sobrescritura, composición y polimorfismo.

4. Estado del trabajo

Actividades completadas

- Organicé el Laboratorio 04 en su propia carpeta.
- Conservé los tres laboratorios anteriores.
- Incorporé las prácticas del material y sus respuestas.
- Implementé el gestor persistente con menú interactivo.
- Implementé almacenamiento JSON protegido y exportación CSV.
- Añadí validaciones y manejo de errores.
- Ejecuté pruebas automatizadas, compilación y una prueba manual de guardado y recuperación.
- Publiqué los archivos y commits en GitHub.
- Actualicé el README principal con las instrucciones para ejecutar los cuatro laboratorios.

Dificultades y soluciones

La principal dificultad fue asegurar que el programa no reemplazara un JSON existente cuando tuviera una sintaxis inválida. Lo solucioné separando la lectura en `almacenamiento.py`, capturando `JSONDecodeError` y notificando el problema sin sobrescribir el archivo. También tuve que separar la lógica del inventario de los reportes para evitar que un cambio en el CSV afectara las operaciones de negocio.

No quedan funcionalidades pendientes para el alcance de este laboratorio. Como mejora futura podría agregar una interfaz gráfica o una base de datos, pero no son requisitos de esta entrega.

5. Seguimiento de commits en GitHub

Estos son los commits publicados y lo que hice en cada etapa:

Commit	Cambio realizado	Motivo y resultado
`b1ba66e`	`feat: agregar persistencia JSON y reportes CSV`	Incorporé las clases, el almacenamiento, los reportes, el menú, las pruebas y las evidencias principales. El gestor quedó funcional y persistente.
`6798f66`	`chore: agregar ejercicios y analisis del laboratorio 04`	Agregué las respuestas de comprobación y análisis del material para dejar documentado el razonamiento técnico.
`0e3ddd9`	`chore: agregar practicas de excepciones del laboratorio 04`	Publiqué los ejercicios de validación, errores, conversiones seguras y separación lógica.
`f6e2ed4`	`chore: agregar practicas de archivos del laboratorio 04`	Publiqué ejercicios de lectura, escritura, rutas, bitácoras y reportes.
`b9673c7`	`chore: agregar practicas JSON y CSV del laboratorio 04`	Publiqué las prácticas de serialización, JSON protegido y archivos CSV.
`fdf2ea3`	`chore: agregar practicas de POO avanzada del laboratorio 04`	Publiqué herencia, composición, sobrescritura y polimorfismo.
`9bcd4ff`	`docs: actualizar README para cuatro laboratorios`	Documenté la estructura completa y los comandos de ejecución del repositorio.

El orden de los commits permite identificar la creación de la solución principal, la incorporación de las prácticas y la actualización de la documentación. Si se necesita revisar un cambio, se puede usar `git log --oneline` y luego `git diff` entre dos hashes.

6. Evidencias de pruebas y comparación con lo esperado

Compilación

Ejecuté `python3 -m compileall -q .` y el resultado fue `compileall_exit=0`. Esto confirma que los módulos no tienen errores de sintaxis.

Pruebas automatizadas

Ejecuté `python3 -m unittest -v test_persistencia.py`. El resultado registrado fue **5 pruebas correctas (`5 OK`)**. Las pruebas verifican registro y operaciones, guardado y recuperación, validaciones, manejo de JSON dañado y generación del CSV.

Prueba manual de persistencia

Registré el producto `P004 - Webcam - Periféricos - $120000.00 - 2 unidades`, guardé y cerré el programa. Después lo abrí nuevamente y el producto apareció en la lista. Esto coincide con el resultado esperado: la información se conserva en `datos/productos.json`.

Resumen

El resumen mostró:

- Productos: `1`
- Unidades: `2`
- Valor total: `$240000.00`

El resultado esperado era `120000 * 2 = 240000`, por lo tanto el cálculo fue correcto.

Reporte CSV

La aplicación informó `Reporte creado en inventario.csv`. La evidencia confirma que la columna `valor_inventario` se genera mediante `precio * cantidad`, que es el cálculo esperado para el valor total de cada producto.

JSON dañado

Al probar un JSON con formato inválido se lanzó `ErrorAlmacenamiento` y el archivo no fue sobrescrito. El resultado esperado era conservar el archivo original para no perder información, y la prueba cumplió ese objetivo.

7. Respuestas a las preguntas de análisis situacional

1. ¿Qué haría si el programa termina inesperadamente y se pierden productos no guardados?

Primero revisaría la consola, el archivo `datos/productos.json` y las rutas que utiliza `almacenamiento.py`. Lo haría porque los productos pueden haber quedado solo en memoria o el programa podría estar leyendo y guardando en carpetas diferentes. Comprobaría que la ruta se construya con `Path(__file__).resolve().parent`, y que el programa guarde después de cada modificación exitosa y también al elegir la opción de salida. Después registraría un producto, forzaría el cierre, volvería a iniciar el programa y compararía la lista con el contenido físico del JSON. Así confirmaría que la corrección realmente conserva los datos.

2. ¿Qué haría si `productos.json` existe, pero tiene una coma faltante?

Capturaría `json.JSONDecodeError` al cargar el archivo, mostraría una advertencia clara y bloquearía el guardado automático sobre ese archivo. No lo trataría como una lista vacía, porque eso destruiría información posiblemente recuperable. Implementaría la lectura dentro de `try/except`, devolvería un error de almacenamiento y mantendría intacto el archivo original. Para comprobarlo, introduciría una coma faltante de forma intencional, iniciaría el programa, verificaría el mensaje y compararía el archivo antes y después. Luego corregiría manualmente el JSON y comprobaría que los productos vuelven a cargarse.

3. ¿Qué haría si aparecen códigos duplicados al recuperar el JSON?

Centralizaría la regla de unicidad en `Inventario`, porque el inventario es el responsable de decidir si un producto puede agregarse. Cargaría los registros primero en una instancia temporal; cada registro pasaría por `Producto.desde_diccionario()` y `agregar_producto()`. Si se repite un código, se produciría un `ValueError` y la colección activa no quedaría parcialmente modificada. Para comprobarlo, crearía un JSON con dos productos `P001`, ejecutaría la carga y verificaría que se informe la inconsistencia y que no quede agregado solo el primer producto.

4. ¿Qué haría si el CSV muestra el precio unitario y no el valor total?

Revisaría `reportes.py`, porque allí se construyen las filas del archivo. Buscaría si la columna `valor_inventario` está recibiendo solamente `p.precio` en lugar de `p.precio * p.cantidad`. Corregiría la expresión y agregaría una prueba de regresión. Por ejemplo, con precio `$45000` y cantidad `5`, el CSV debe mostrar `225000.0`. Ejecutaría nuevamente la exportación y leería la fila generada para comparar el resultado con el cálculo manual.

5. ¿Qué haría si una modificación del guardado rompe la carga?

Usaría el historial de GitHub para localizar el último commit funcional con `git log --oneline` y compararía los cambios mediante `git diff <hash_anterior> <hash_actual>`. Revisaría especialmente los nombres de las claves que produce `a_diccionario()` y las que espera `desde_diccionario()`, porque cambiar `codigo` por `cod`, por ejemplo, puede provocar un `KeyError`. Corregiría la estructura del dato, ejecutaría una prueba completa de registrar, guardar, reiniciar y recuperar, y documentaría la solución con un commit como `fix: corregir claves serializables en almacenamiento`. De esta forma quedaría registrada la causa, la solución y el resultado comprobado.

8. Conclusión

Con este laboratorio completé la evolución del gestor de inventario hasta una solución modular y persistente. Aprendí que guardar datos no consiste solo en escribir un archivo: también es necesario validar, manejar errores, proteger la información existente y comprobar que la lectura reconstruya correctamente los objetos. El repositorio público contiene el código, las prácticas, las evidencias y el historial de commits que respaldan el proceso.